CSE 242 — Assignment 3 Report

Question 1: Hard-margin linear SVM

(a) Support vectors

Plot: `svm_data.png`. The positive points are 1 (1,1), 2 (1,2), 3 (2,1); the negative points are 4 (0,0), 5 (1,0), 6 (0,1).

The support vectors are **points 1, 5, and 6**. Point 1 is the positive point nearest the negative class: its distance to points 5 and 6 is 1, and to point 4 is $\sqrt{2}$. Points 5 and 6 are the negative points nearest the positive class. Points 2 and 3 are farther from the negatives (distance $\geq \sqrt{2}$ from 5 and 6), and point 4 is shielded behind 5 and 6, so none of them constrains the margin.

(b) Boundary and margin

Support vectors lie exactly on the margin, so $y(w \cdot x + b) = 1$ for each:

- Point 1: $w1 + w2 + b = 1$
- Point 5: $w1 + b = -1$
- Point 6: $w2 + b = -1$

Subtracting (2) from (1) gives $w2 = 2$, and subtracting (3) from (1) gives $w1 = 2$. Then $b = -1 - w1 = -3$.

$w = (2, 2)$, $b = -3$, so the boundary is **$2x1 + 2x2 - 3 = 0$** (equivalently $x1 + x2 = 1.5$).

Check on the other points: point 2 $\rightarrow$ $2+4-3 = 3 \geq 1$; point 3 $\rightarrow$ $3 \geq 1$; point 4 $\rightarrow$ $-3 \leq -1$. All constraints hold.

Dual check: $w = \Sigma \alpha_i y_i x_i$ requires $\alpha1(1,1) - \alpha5(1,0) - \alpha6(0,1) = (2,2)$ with $\Sigma \alpha_i y_i = 0$. Symmetry gives $\alpha5 = \alpha6 = a$, $\alpha1 = 2 + a$, and $\alpha1 = 2a$ gives $a = 2$, so $\alpha = (4, 2, 2)$. All are positive, so the KKT conditions are satisfied and this is the maximum-margin solution.

Geometric margin (distance from the boundary to the closest training points) = $1/\|w\| = 1/(2\sqrt{2}) =$ **$\sqrt{2}/4 \approx 0.3536$**. The full width between the two supporting boundaries is $2/\|w\| \approx 0.7071$ (reported separately).

Question 2: K-means

(a) Implementation

- `update_assignments` computes the Euclidean distance from every point to every centroid and takes `argmin`. **Ties** go to the lowest centroid index, since `argmin` returns the first minimum.
- `update_centroids` sets each nonempty cluster's centroid to the mean of its members. **Empty clusters** keep their previous centroid. The input array is copied, not modified. This also covers duplicate initial centroids, since the lower-indexed duplicate wins every tie and the other becomes an empty cluster that stays put.

(b) Results

Data: `make_blobs(n_samples=4000, centers=4, cluster_std=0.60, random_state=0)`, seed 0, 100 iterations. Plots are in `results/kmeans/` (`kmeans_k2.png`, `kmeans_k3.png`, `kmeans_k4.png`, `kmeans_k6.png`, `kmeans_k10.png`, plus `reference.png` showing the true blobs, which were not used for clustering).

k	Cluster sizes
2	1017, 2983
3	1008, 1992, 1000
4	1000, 998, 1002, 1000
6	1000, 299, 997, 407, 319, 978
10	365, 350, 253, 261, 988, 997, 123, 213, 285, 165

Discussion. The data has four well-separated groups of about 1000 points each. At k = 2 and k = 3, groups are forced to merge (the 1992-point cluster at k = 3 is two blobs combined). At k = 4 the clusters match the four generating groups almost exactly, with sizes about 1000 each. For k > 4 there are more centroids than groups, so some true blobs are split into arbitrary sub-clusters (at k = 6 three clusters of about 1000 points remain, and the rest of the data is split into pieces of 299–407 points). Within-cluster variance keeps falling as k grows, so the lower error at larger k does not mean a better clustering. The right k is revealed by the structure of the data, not by the objective alone. Results also depend on the random initialization, since centroids are sampled from the data.

Question 3: Noisy-label SVM and K-NN repair

Setup: one fixed clean training set and one fixed clean test set (1000 positives from $N((1,1), I)$, 1000 negatives from $N((-1,-1), 3I)$ each), seed 0. For each of 20 trials, labels were corrupted from the original labels (positive flipped with probability 0.35, negative with 0.20, each at most once). An RBF SVM ($C = 1$, $\gamma = 0.01$) was trained on the noisy labels (baseline) and, separately, on K-NN-repaired versions of the same noisy labels. Repair uses 5 nearest other training points (Euclidean), takes the majority of their noisy labels, and computes all repairs from the original noisy array. Evaluation used clean test labels. Per-trial results are in `results/noise_comparison.csv`.

Method	Mean test accuracy (20 trials)
Baseline (noisy labels)	0.8280
K-NN repaired labels	0.8644
Difference (K-NN − baseline)	+0.0364

Discussion. K-NN repair helped in this experiment, raising mean accuracy by about 3.6 points. It can help because the noise is independent of position: a point's neighbors are mostly of its true class, and five neighbors' noisy labels will usually still carry a majority for the true label, even at 35% noise, so many flipped labels are corrected. It can hurt where the classes overlap (the negative class has a large variance, so many negatives sit in the positive region), because neighbor majorities there reflect the other class and repair can turn correct labels into wrong ones. Majority voting also smooths away genuine local structure and pushes labels toward the locally dominant class. The improvement is therefore not guaranteed and depends on the noise rate and the class overlap. The result is from a single fixed dataset, so it is not a general claim.